QSAR + REINVENT Pipeline Overview
Simple, Process-Focused Project Presentation

Based on the comprehensive report and repository structure documents

QSAR Project Team

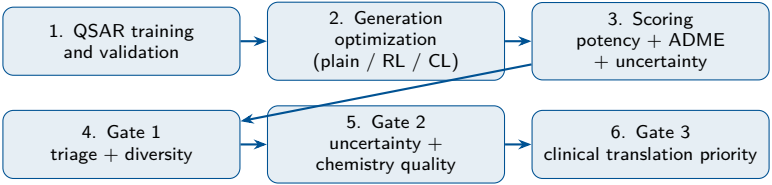
Computational Drug Discovery

March 15, 2026

What this presentation covers

1. QSAR to translation walkthrough
2. QSAR Modeling (Detailed)
3. REINVENT4 Generation (Detailed)
4. Gate 1 (Detailed)
5. Gate 2 (Detailed)
6. Gate 3 (Detailed)
7. Why this project
8. Key terms (plain language)
9. REINVENT 4 in this project
10. Process schema
11. How scoring is carried out (math)
12. Repository map and outputs
13. Practical usage

Pipeline order used in this project



Why this ordering

Learn from known data first (QSAR), optimize generation next, then use staged risk filters before selecting compounds for lab planning.

Data acquisition $\rightarrow$ deployment lifecycle

1) Data acquisition

Experimental series data (C/E + historical labels), SMILES cleaning, canonicalization, and deduplication.

2) Modeling

Descriptor generation + model selection for potency prediction. Best potency model: **RDKit + KNN**.

3) Evaluation

Held-out validation on Series C and prospective ranking on Series E.

4) Testing (in-silico pipeline testing)

REINVENT generation campaigns, automated scoring, and stage-gated filtering (Gate 1 $\rightarrow$ 2 $\rightarrow$ 3).

5) Deployment / operational use

Final ranked workbook is produced for chemistry/biology teams to choose wet-lab candidates and plan experiments.

Key current results

Series C: MAE **0.118**, $R^2 = \mathbf{0.887}$
Gate flow: **200** $\rightarrow$ **46** $\rightarrow$ **34**

Step 1: QSAR model training and key result

How QSAR is built

1. Clean and standardize molecular structures.
2. Compute descriptor families (RDKit/ECFP/MACCS/Mordred).
3. Train multiple regressors and compare on held-out data.
4. Keep best model for primary IC_{50} prediction.

Key performance (current artifact)

Best model:
RDKit + KNN
$R^2 \approx 0.887$

Additional positive-R^2 models used for uncertainty:

- ECFP_r3 + KNN
- RDKit + Decision Tree
- RDKit + LGBM
- RDKit + Extra Trees

Where QSAR test outputs are saved (Series C/E)

Primary C/E prediction files

`outputs/qsar/core_predictions/SeriesC_test_with_predictions.csv`

`outputs/qsar/core_predictions/SeriesE_with_predictions.csv`

Manifest files

`outputs/qsar/manifests/qsar_outputs_manifest.csv`

`outputs/qsar/manifests/additional_qsar_outputs_manifest.csv`

What each contains

- Canonical structures and identifiers
- Predicted pIC_{50} and predicted IC_{50} (μM)
- Source/series metadata and downstream analysis fields

Series C and E testing: key QSAR outputs

Series C (test set)

- Total rows: **6**
- Rows with measured pIC_{50}: **5**
- MAE (pIC_{50}): **0.118**
- R^2: **0.887**

Series E (prediction set)

- Total rows predicted: **6**
- Predicted pIC_{50} range: **4.467 to 5.132**
- Predicted IC_{50} range: **7.38 to 34.11 μM**

Interpretation for presentation

Series C confirms model quality on known test compounds; Series E provides prospective potency ranking for compounds without direct fitted labels.

Step 2: Generation optimization (plain, RL, CL)

How optimization works

- **Plain**: broad exploration from prior model.
- **RL**: sample → score → update agent.
- **CL**: staged objectives from easier to stricter targets.

Why CL is important

More stable learning, fewer invalid molecules, better quality at convergence.

Used in this project

Families/modes were run across campaigns, then all sampled outputs were passed through one consistent scoring and gating workflow.

Typical path:

`reinvent_integration/results/`

`<family_mode>/samples.csv`

Step 3: Scoring and ranking logic

Scoring stack

Validate SMILES → descriptors → potency prediction → ADME/risk enrichment → uncertainty estimation → gate-ready output fields.

Gate 1 triage equation

$$\text{triage} = 0.35P + 0.15S + 0.15L + 0.10M + 0.10H + 0.15T$$

P: potency, S: solubility, L: cLogP window,
M/H: clearance terms, T: half-life.

Scored output location

`outputs/generated/<family>/<mode>/`
CSV + Excel with structures and fields used by gates.

How uncertainty is controlled in ranking

Uncertainty calculation

Up to 5 positive-R^2 models predict each molecule.

$$\mu = \frac{1}{k}\sum_{i=1}^{k}\hat{y}_i, \quad \sigma = \sqrt{\frac{1}{k-1}\sum_{i=1}^{k}(\hat{y}_i - \mu)^2}$$

Lower σ means higher confidence.

Control mechanism

- Primary potency remains best model output.
- Gate 2 penalizes high disagreement.
- Confidence columns are carried into final ranking.

Practical effect

Reduces false positives that look strong in one model but unstable across the ensemble.

Step 4: Clinical translation stage (Gate 3)

What Gate 3 optimizes

- Toxicity risk (e.g., hERG, DILI, AMES proxies)
- DDI/CYP risk (interaction liability)
- Absorption support (HIA/bioavailability proxies)
- Final experimental priority score

Current outputs

`outputs/hybrid_stage/gate3_target_free_translational/translational_candidates.xlsx`

Current pass counts

Gate 1: 200 shortlisted
Gate 2: 46 passes
Gate 3: 34 passes

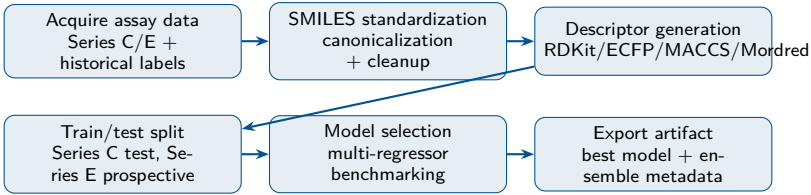
What was done and what you can present

1. Built QSAR with best-performing potency model and ensemble uncertainty.
2. Ran REINVENT generation campaigns across families and optimization modes.
3. Scored all generated molecules with one consistent scoring pipeline.
4. Applied stage-gated ranking (Gate 1 $\rightarrow$ Gate 2 $\rightarrow$ Gate 3).
5. Produced ranked Excel outputs for medicinal-chemistry review and lab planning.

Final narrative

QSAR learns, **REINVENT explores/optimizes**, **gates control risk**, and **Gate 3 translates to action**.

QSAR modeling flowchart: data acquisition to model artifact



QSAR modeling details: what is trained and selected

Candidate models tested

- KNN, Decision Tree, Extra Trees
- Gradient boosting family (e.g., LGBM)
- Descriptor families compared side by side
- Selection by held-out predictive quality

Selected setup (current)

- Primary potency model: **RDKit + KNN**
- Key metric: $R^2 \approx 0.887$
- Backup positive-R^2 models kept for uncertainty
- Artifact path:

  `reinvent_integration/artifacts/qsar_best_model.joblib`

QSAR evaluation and test outputs (Series C and E)

Series C (test evaluation)

- Total rows: **6**
- Measured pIC_{50} rows: **5**
- MAE (pIC_{50}): **0.118**
- R^2: **0.887**

Series E (prospective predictions)

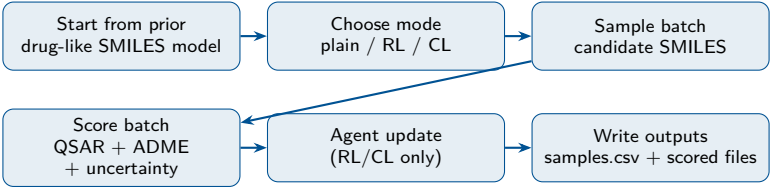
- Predicted rows: **6**
- Predicted pIC_{50}: **4.467 to 5.132**
- Predicted IC_{50}: **7.38 to 34.11 μM**

Output files

`outputs/qsar/core_predictions/SeriesC_test_with_predictions.csv`

`outputs/qsar/core_predictions/SeriesE_with_predictions.csv`

REINVENT4 generation flowchart



REINVENT4 modes and optimization details

Mode	How optimization behaves
Plain	No policy update; broad exploration from prior distribution.
RL	Iterative policy optimization from reward signal after scoring each batch.
CL	Multi-stage objective: easier constraints first, stricter objectives later.
Mol2Mol/LibINVENT/LinkINVENT	Family-specific generation constraints layered on top of training mode.

Why curriculum learning was used

It improves stability and validity while pushing toward higher-quality, multi-objective molecules under realistic constraints.

REINVENT4 project execution and outputs

What was run

- Multiple generation families and modes
- Automated score pass after each sampling output
- Unified downstream gate ranking across all runs

Where outputs are saved

Raw samples:

`reinvent_integration/results/<family_mode>/samples.csv`

Scored results:

`outputs/generated/<family>/<mode>/`

Generation activity scope in this project (not just 3 modes)

Model families orchestrated in campaigns

reinvent, reinvent_pubchem, pepinvent, mol2mol_similarity, mol2mol_high_similarity, mol2mol_medium_similarity, mol2mol_scaffold, mol2mol_scaffold_generic, mol2mol_mmp, libinvent, libinvent_transformer_pubchem, linkinvent, linkinvent_transformer_pubchem.

Operational implication

Each family was executed across plain/RL/CL flows with post-scoring, so the generation program is a **matrix of campaigns**, not a single run.

Generation campaign matrix: families × optimization modes

Family group	Plain	RL	CL
REINVENT / REINVENT_PubChem	✓	✓	✓
PepINVENT	✓	✓	✓
Mol2Mol (sim/high/medium/scaffold/mmp)	✓	✓	✓
LibINVENT (+ transformer prior)	✓	✓	✓
LinkINVENT (+ transformer prior)	✓	✓	✓

Why this matrix matters

It separates exploration (plain), exploitation (RL), and robust staged optimization (CL), while covering multiple molecular design tasks (de novo, analoging, linking, decorating).

REINVENT / REINVENT_PubChem (✓✓✓)

What this family contributes

- Core de novo generation backbone
- REINVENT_PubChem adds broader prior chemistry exposure
- Strong for novelty-first exploration

Modes in this project

Plain: baseline de novo sampling
RL: reward-guided potency/ADME optimization
CL: staged objective tightening for stable convergence

Concrete run artifacts

Config examples:

`sample_reinvent_plain.toml`

`rl_reinvent.toml` / `cl_reinvent.toml`

`sample_reinvent_rl.toml` / `sample_reinvent_cl.toml`

Raw outputs:

`reinvent_integration/results/reinvent_<mode>/samples.csv`

Scored outputs:

`outputs/generated/reinvent/<mode>/`

PepINVENT (✓✓✓)

What this family contributes

- Peptide-like sequence/structure generation focus
- Complements small-molecule-focused families
- Adds modality diversity to candidate pool

Modes in this project

Plain: broad peptide-like proposal space
RL: prioritize higher reward peptide candidates
CL: staged constraints to improve quality stability

Concrete run artifacts

Config examples:

`sample_pepinvent_plain.toml`

`rl_pepinvent.toml` / `cl_pepinvent.toml`

`sample_pepinvent_rl.toml` / `sample_pepinvent_cl.toml`

Raw outputs:

`reinvent_integration/results/pepinvent_<mode>/samples.csv`

Scored outputs:

`outputs/generated/pepinvent/<mode>/`

Mol2Mol (sim/high/medium/scaffold/mmp) (✓✓✓)

What this family contributes

- Controlled analog generation around references
- Multiple transformation regimes: similarity, scaffold, MMP
- Balances novelty with medicinal chemistry continuity

Modes in this project

Plain: analog exploration without policy updates
RL: optimize transformations toward reward
CL: staged analog objective tightening per regime

Concrete run artifacts

Examples:

`sample_mol2mol_similarity_plain.toml`

`rl_mol2mol_high_similarity.toml`

`cl_mol2mol_scaffold.toml`

Raw outputs:

`reinvent_integration/results/mol2mol_<variant>_<mode>/`

Scored outputs:

`outputs/generated/mol2mol/<variant>/<mode>/`

LibINVENT (+ transformer prior) (✓✓✓)

What this family contributes

- Scaffold decoration/library-style expansion
- Useful for R-group exploration at fixed cores
- Transformer prior variant broadens syntactic diversity

Modes in this project

Plain: enumerate baseline decoration ideas
RL: improve decoration reward profile
CL: staged decoration quality constraints

Concrete run artifacts

Config examples:

`sample_libinvent_plain.toml`

`rl_libinvent.toml` / `cl_libinvent.toml`

`sample_libinvent_transformer_pubchem_cl.toml`

Raw outputs:

`reinvent_integration/results/libinvent_<mode>/samples.csv`

`reinvent_integration/results/libinvent_transformer_pubche`

LinkINVENT (+ transformer prior) (✓✓✓)

What this family contributes

- Linker design between anchors/fragments
- Critical for fragment-joining and geometry-aware assembly
- Transformer prior variant increases linker novelty breadth

Modes in this project

Plain: linker idea exploration
RL: optimize linker candidates by reward
CL: staged linker constraint tightening

Concrete run artifacts

Config examples:

`sample_linkinvent_plain.toml`

`rl_linkinvent.toml` / `cl_linkinvent.toml`

`sample_linkinvent_transformer_pubchem_cl.toml`

Raw outputs:

`reinvent_integration/results/linkinvent_<mode>/samples.cs`

`reinvent_integration/results/linkinvent_transformer_pubch`

Generation config taxonomy used in repository

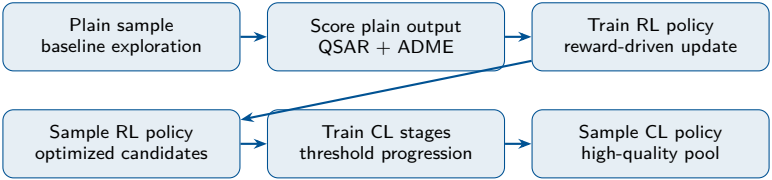
Core config types

- `sample_<family>_plain.toml`
- `rl_<family>.toml`
- `sample_<family>_rl.toml`
- `cl_<family>.toml`
- `sample_<family>_cl.toml`

Variant config sets

- `reinvent_integration/configs/`
- `reinvent_integration/configs_fast/`
- `reinvent_integration/configs_strict/`
- Nitro campaign-specific configs

Generation execution lifecycle (per family)



How run orchestration is implemented

Controller scripts

```
reinvent_integration/run_all_generation.py
reinvent_integration/run_all_generation_fast.py
reinvent_integration/run_all_generation_strict.py
reinvent_integration/run_nitro_bioisostere_campaign.py
```

Automated per-family sequence

For each family: plain sample → score, RL train → RL sample → score, CL train → CL sample → score, then merge into hybrid stage-gate pipeline.

Diverse generation activities: family-specific intent

Exploration and analoging

- REINVENT / REINVENT_PubChem: broad de novo novelty
- Mol2Mol similarity/high/medium: controllable analog search
- Mol2Mol scaffold/scaffold_generic: scaffold-preserving design
- Mol2Mol MMP: matched-pair style transformations

Assembly and decoration tasks

- PepINVENT: peptide-like sequence/structure design
- LinkINVENT: linker optimization between anchors
- LibINVENT: scaffold decoration libraries
- Transformer-PubChem variants: broader prior transfer

Optimization depth: what RL and CL improve over plain

Aspect	Practical behavior in pipeline
Potency targeting	RL/CL shift sampled distribution toward higher QSAR-ranked molecules.
ADME alignment	Objective shaping increases fraction within preferred ADME windows.
Feasibility control	CL staging reduces abrupt jumps that produce unstable candidates.
Gate readiness	RL/CL outputs generally show better downstream Gate 1 and Gate 2 conversion.

Generation run variants: default, fast, strict, campaign-specific

Default/full campaign

Broad model matrix, full scoring stack, standard gate thresholds for ranked export.

Strict variant

Tighter constraints / stricter progression, useful for high-confidence candidate narrowing.

Fast variant

Reduced runtime profile for rapid iteration and sanity checks.

Nitro bioisostere campaign

Focused chemistry campaign with dedicated configs and logs.

Generation logs: what information is captured

Log locations and examples

`reinvent_integration/logs/run_all_generation.log`
`reinvent_integration/logs/run_all_generation_fast.log`
`reinvent_integration/logs/run_all_generation_strict.log`
Family logs: `<family>_<mode>_<train/sample>_*.log`

Operational signals extracted

- step completion/failure status for each run
- training progress and sampling completion
- mode-specific checkpoints and campaign diagnostics

Generation output hierarchy and handoff to scoring

Raw generation outputs

`reinvent_integration/results/<family>_<mode>/s`
plus campaign-level artifacts/checkpoints in results tree.

Scored generation outputs

`outputs/generated/<family>/<mode>/`
standardized scored files used by Gate 1 input pooling.

End-to-end linkage

Generation activity → scoring normalization → Gate 1 ranking → Gate 2 refinement → Gate 3 translational shortlist.

Generation coverage summary for presentation narrative

Core message to stakeholders

The project performed a **diversified generation program** spanning de novo design, analoging, scaffold-conditioned transformation, linker design, and library decoration, each optimized via plain/RL/CL and then evaluated in one consistent scoring + gate framework.

Why this is stronger than a single-generator story

It reduces model-family bias, increases chemical search breadth, and produces more robust downstream candidates for translational prioritization.

REINVENT4 generation families: what each generates

Family	Generation behavior and practical use
REINVENT (de novo)	Generates full molecules from learned prior; best for broad exploration.
Mol2Mol	Starts from reference scaffold/seed and proposes close-but-improved analogs.
LinkINVENT	Generates linkers to connect fragments while preserving anchor context.
LibINVENT	Enumerates decoration vectors around a scaffold under reaction-like constraints.
Transformer-PubChem variants	Broader pretraining bias for novelty and chemical grammar robustness.

Project strategy

Use multiple families in parallel: broad exploration (de novo) + local optimization (scaffold/fragment-driven families), then merge all outputs into one scoring/gating path.

Plain generation details (exploration mode)

What happens in plain mode

- Agent weights are fixed (no optimization loop)
- Batched sampling from prior policy
- Strongly useful for baseline diversity
- Fast throughput for first-pass idea space

How outputs are used

- Post-scored by QSAR + ADME pipeline
- Feeds Gate 1 diversity-aware shortlist
- Acts as discovery baseline vs RL/CL gains

Reinforcement learning generation: optimization loop

Sample molecules from current policy → Compute reward multi-objective score → Update policy increase expected reward → Resample and repeat until step budget

Optimization objective (conceptual)

$$J(\theta) = \mathbb{E}_{x \sim \pi_\theta}[R(x)] \quad \Rightarrow \quad \theta \leftarrow \theta + \eta \nabla_\theta J(\theta)$$

Where $R(x)$ combines potency, ADME, and feasibility/risk constraints.

Curriculum learning generation: staged optimization

Why curriculum is stronger than single-stage RL

1. Stage 1: satisfy validity/basic chemistry and minimal potency.
2. Stage 2: tighten potency + key ADME windows.
3. Stage 3: enforce full objective stack before final sampling.

This reduces optimization collapse and improves final feasible hit-rate.

Progression rule (conceptual)

Advance stage when:

$$\bar{R}_{\text{window}} \geq \tau_s$$

where τ_s is stage threshold.

Reward construction used for REINVENT optimization

Composite reward structure

$$R = w_p f_{\text{potency}} + w_s f_{\text{solubility}} + w_l f_{\log P} + w_m f_{\text{microsomal}} + w_h f_{t_{1/2}} - w_u f_{\text{uncertainty}} - w_t f_{\text{tox risk}}$$

Positive terms drive efficacy/developability; penalty terms suppress unstable or risky candidates.

Important practical point

Generation optimization and gate triage are aligned but not identical: reward guides learning dynamics, gates enforce strict deployment filters.

How to read REINVENT optimization quality in this project

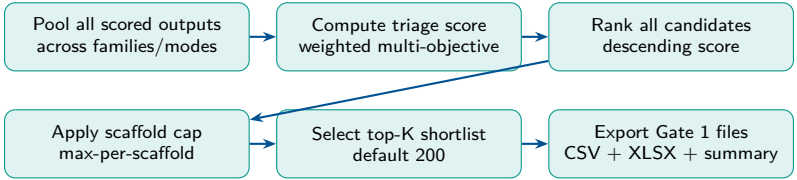
Healthy optimization signals

- Mean score trend increases over steps
- Valid SMILES fraction stays high
- Diversity does not collapse too early
- Top candidates survive Gate 1 and Gate 2

Warning patterns to monitor

- Score spike with low diversity (mode collapse)
- Rising uncertainty in high-score molecules
- ADME out-of-window drift at later steps
- Weak Gate 2/3 conversion ratio

Gate 1 flowchart: triage and diversity shortlist



Gate 1 details: scoring and output artifacts

Triage equation

$$\text{triage} = 0.35P + 0.15S + 0.15L + 0.10M + 0.10H + 0.15T$$

Potency is weighted highest; ADME balance and stability terms control quality.

Gate 1 outputs

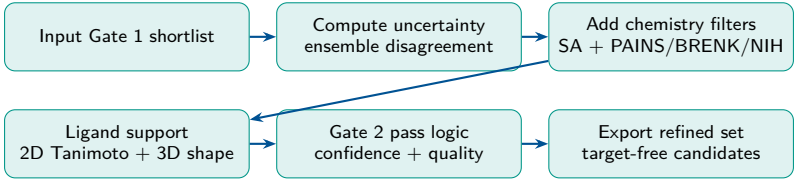
`outputs/hybrid_stage/gate1/gate1_master_ranked.csv`

`outputs/hybrid_stage/gate1/gate1_shortlist.csv`

`outputs/hybrid_stage/gate1/gate1_shortlist.xlsx`

`outputs/hybrid_stage/gate1/gate1_summary.json`

Gate 2 flowchart: uncertainty and chemistry refinement



Gate 2 details: uncertainty control and outputs

Uncertainty control mechanism

Primary potency stays the best model prediction; up to five positive-R^2 models are used to compute disagreement (σ). High-σ molecules are deprioritized or filtered.

Gate 2 output files

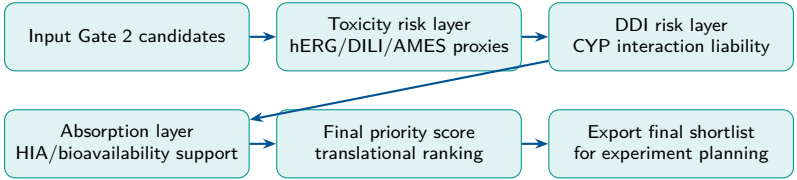
`outputs/hybrid_stage/gate2_target_free_refinement/target_free_candidates.csv`

`outputs/hybrid_stage/gate2_target_free_refinement/target_free_candidates.xlsx`

Current count

Gate 2 passes: **46**

Gate 3 flowchart: translational prioritization



Gate 3 details: outputs and deployment handoff

Final Gate 3 artifacts

`outputs/hybrid_stage/gate3_target_free_translational/translational_candidates.csv`

`outputs/hybrid_stage/gate3_target_free_translational/translational_candidates.xlsx`

`outputs/hybrid_stage/gate3_target_free_translational/experimental_shortlist_template.csv`

Current count

Gate 3 passes: **34**

Deployment meaning in this project

"Deployment" = operational delivery of ranked candidates to medicinal chemistry / biology for wet-lab testing decisions.

Problem we are solving

Goal

Start from many AI-generated molecules and end with a small, high-quality shortlist for laboratory testing.

- We do **not** have a reliable protein target structure for physics-heavy simulation.
- So we use a **target-free, data-driven pipeline**.
- We combine potency prediction, ADME quality, uncertainty, safety risk, and diversity before selecting final candidates.

Business rationale

Reduce expensive false positives early, while keeping chemical diversity high.

Core idea in one line

Generate many molecules $\rightarrow$ score quickly $\rightarrow$ refine confidently $\rightarrow$ prioritize translationally

Key terms everyone should know

Term	Simple meaning
SMILES	Text format for molecules (a way to write chemistry as strings).
QSAR	Model that predicts activity/properties from structure.
IC_{50}	Potency: lower value means stronger molecule.
pIC_{50}	Log version of IC_{50}: higher value means stronger molecule.
ADME	Absorption, distribution, metabolism, excretion properties.
Murcko scaffold	Core ring skeleton used to track diversity.
Uncertainty	How much multiple good models disagree on prediction.

Generation terms used in this project

Term	Simple meaning
REINVENT (plain)	Generate molecules directly from pretrained chemistry language model.
RL mode	Reinforcement learning: reward molecules with better predicted profile.
CL mode	Curriculum learning: optimize in staged difficulty.
Mol2Mol	Create analogs from seed molecules.
LibINVENT	Decorate a scaffold with substituents.
LinkINVENT	Connect fragments with designed linker.

How REINVENT 4 generation modes work

Mode	How it generates molecules
Plain (prior sampling)	Samples directly from pretrained chemical language model for broad exploration.
RL (reinforcement learning)	Generates a batch, scores it, then updates the agent to prefer higher-scoring molecules.
CL (curriculum learning)	Runs optimization in stages: easier objective first, then stricter objective later.
Mol2Mol	Starts from known molecules and proposes close analogs.
LibINVENT	Keeps a core scaffold and decorates attachment points with substituents.
LinkINVENT	Connects two fragments with generated linkers.

How curriculum learning optimization is done

Optimization sequence

1. Start from prior model (chemically valid baseline).
2. Stage 1 objective: easier constraints (validity + broad quality).
3. Stage 2 objective: stricter score (potency/ADME/uncertainty).
4. At each step: sample $\rightarrow$ score $\rightarrow$ update agent.
5. Stop at max steps or convergence, then sample final set.

Why curriculum helps

- More stable optimization
- Fewer invalid molecules
- Better quality-diversity tradeoff
- Less mode collapse

How REINVENT 4 was used in this project

What was done

Multiple generation families were run (including Mol2Mol, PepINVENT, LibINVENT, LinkINVENT and transformer variants) in one or more training modes (plain, RL, CL), then each sampled output was scored by the same QSAR + ADME + uncertainty pipeline.

Execution flow

1. Run REINVENT campaign config
2. Write `samples.csv`
3. Score via `score_generated_smiles_adme.py`
4. Save scored CSV/XLSX

Where files are

- Raw generations: `reinvent_integration/results/`
- Scored outputs: `outputs/generated/<family>/<mode>/`
- Gate outputs: `outputs/hybrid_stage/`

REINVENT outputs that feed decision-making

Per-run output

Each campaign produces generated SMILES plus scored tables containing potency, ADME, uncertainty, and source metadata.

Cross-run integration

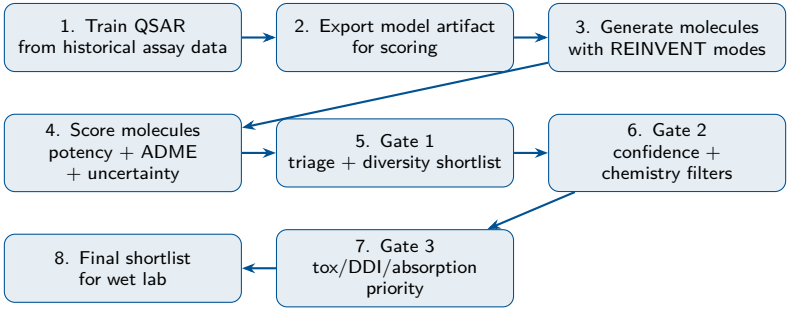
All scored runs are pooled and moved through:

- **Gate 1**: triage + scaffold-diverse top shortlist
- **Gate 2**: confidence and chemistry refinement
- **Gate 3**: translational ranking for experiment planning

Final deliverable

A ranked candidate workbook with structures and rationale-ready fields, used by chemistry and biology teams for lab selection.

End-to-end process schema



What each gate does (simple view)

Gate 1

Fast triage

- Potency + ADME
- Composite triage score
- Scaffold-diverse top 200

Gate 2

Confidence & quality

- QSAR uncertainty
- Synthetic accessibility
- PAINS/BRENK/NIH alerts
- 2D/3D ligand similarity

Gate 3

Translational priority

- Toxicity risk
- CYP/DDI risk
- Absorption support
- Final experiment rank

How scoring is carried out (step-by-step)

1. **Validate SMILES** (remove invalid molecules).
2. **Compute descriptors** from structure.
3. **Predict potency** with best QSAR model.
4. **Estimate uncertainty** from up to 5 positive-R^2 models.
5. **Add ADME / risk fields** (solubility, lipophilicity, clearance).
6. **Apply gate logic** for ranking and filtering.
7. **Export CSV + Excel** with structures for human review.

Outcome

Every candidate has machine-scored evidence + a readable review format, so computational and chemistry teams can make the same decision from the same table.

Scoring mathematics: Gate 1 triage score

Composite triage function

$$\text{triage} = 0.35\,P + 0.15\,S + 0.15\,L + 0.10\,M + 0.10\,H + 0.15\,T$$

with P=potency, S=solubility, L=cLogP window, M=microsomal clearance, H=hepatocyte clearance, T=half-life.

Normalization

Higher-is-better:

$$x_{norm} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Lower-is-better features are inverted after scaling.

Why these weights? (rationale)

- **0.35 Potency**: strongest requirement; non-potent compounds are rarely useful.
- **0.30 Solubility + cLogP**: captures absorption/permeability balance.
- **0.20 Clearance terms**: avoids compounds that disappear too quickly in vivo.
- **0.15 Half-life**: rewards exposure duration without dominating the score.

Important interpretation

This is a **triage utility score**, not a perfect biological truth. It is designed to rank candidates consistently for decision-making.

How uncertainty is computed

Model strategy

Keep the **best $\mathbf{IC}_{50}$** model as primary prediction, then evaluate up to **five positive-R^2** models for the same molecule.

Uncertainty proxy

Let predictions be $\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_k$, $k \leq 5$.

$$\mu = \frac{1}{k}\sum_{i=1}^{k}\hat{y}_i, \quad \sigma = \sqrt{\frac{1}{k-1}\sum_{i=1}^{k}(\hat{y}_i - \mu)^2}$$

Larger σ means lower confidence.

Why it helps

Prevents over-prioritizing molecules that look strong in only one model.

Who carries out scoring? (roles in process)

Role	What they do
REINVENT engine	Generates candidate SMILES across modes (plain/RL/CL).
QSAR scoring script	Computes potency + ADME + uncertainty fields.
Gate script	Applies triage math, diversity selection, Gate 2/3 ranking.
Medicinal chemist	Reviews computed outputs for structural quality and synthesis practicality.
DMPK/safety scientist	Interprets computed tox/DDI/absorption risk outputs for go/no-go decisions.
Project lead	Approves final shortlist for experiment.

Repository structure (current)

Code folders

- `qsar_core/` core modules
- `reinvent_integration/` generation + scoring + gates
- `scripts/` runnable scripts
- `config/` environment files

Data & outputs

- `data/raw/` input dataset
- `outputs/generated/` scored generations
- `outputs/hybrid_stage/` gate outputs
- `docs/` plain-language documentation

Main output files used in decisions

1) Gate 1 shortlist (diversity + triage)

`outputs/hybrid_stage/gate1/ gate1_shortlist.xlsx`

2) Gate 2 refinement (confidence + chemistry quality)

`outputs/hybrid_stage/gate2_target_free_refinement/ target_free_candidates.xlsx`

3) Gate 3 final ranking (translational priority)

`outputs/hybrid_stage/gate3_target_free_translational/ translational_candidates.xlsx`

How to run the pipeline (practical commands)

From repository root

- Export / refresh model artifact:

  ```
  python reinvent_integration/export_qsar_model.py
  ```

- Run generation pipeline:

  ```
  python scripts/run_pipeline.py
  ```

- Build stage-gated rankings:

  ```
  python reinvent_integration/run_hybrid_stage_gates.py
  ```

Preset options

```
python scripts/run_pipeline.py --generation-preset fast|balanced|diverse
```

Decision logic summary

1. Generate broad chemical space.
2. Score with potency + ADME + uncertainty.
3. Keep diversity while ranking quality.
4. Remove uncertain / high-liability / hard-to-make molecules.
5. Prioritize by translational risk and experiment value.

Why this works in no-target settings

It is fast, auditable, and aligned with real project decision gates, even when structure-based physics methods are unavailable.

Thank you

Questions?

Reference docs used:

`docs/REPO_STRUCTURE_AND_OUTPUTS.txt`

`docs/TERMS_AND_WORKFLOW_EXPLAINED.txt`